Pandas is a powerful library in Python used for data manipulation and analysis. It provides data structures like DataFrame and Series that are efficient for handling structured data, such as CSV files, Excel sheets, SQL tables, and more.

Data Structures:

DataFrame: A 2-dimensional labeled data structure with columns of potentially different types, similar to a spreadsheet or SQL table. It is the primary pandas data structure used in data science.

Series: A one-dimensional labeled array capable of holding any data type, similar to a column in a DataFrame.

```
import pandas as pd
import numpy as np

# Creating a dummy dataset
data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'David', 'Eve'],
    'Age': [25, 30, 35, 40, 45],
    'Gender': ['F', 'M', 'M', 'M', 'F'],
    'Math_Score': [85, 92, 78, 95, 80],
    'Science_Score': [90, 88, 82, 96, 85],
    'English_Score': [88, 85, 90, 92, 78],
    'History_Score': [85, 82, 88, 90, 92]
}

df = pd.DataFrame(data)
df
```

```
      Name  Age Gender  Math_Score  Science_Score  English_Score  \
0    Alice   25      F          85             90             88
1      Bob   30      M          92             88             85
2  Charlie   35      M          78             82             90
3    David   40      M          95             96             92
4      Eve   45      F          80             85             78

   History_Score
0             85
1             82
2             88
3             90
4             92
```

Step 2: Selecting a Subset of the DataFrame Let's select students who scored above 85 in Math:

```
subset = df[df['Math_Score'] > 85]
subset
```

```
    Name  Age Gender  Math_Score  Science_Score  English_Score
History_Score
1    Bob   30      M          92             88             85
82
3  David   40      M          95             96             92
90
```

Creating New Columns Derived from Existing Columns Let's create a new column called "Total_Score" which is the sum of all subject scores:

```
df['Total_Score'] = df['Math_Score'] + df['Science_Score'] +
df['English_Score'] + df['History_Score']
df
```

```
      Name  Age Gender  Math_Score  Science_Score  English_Score  \
0    Alice   25      F          85             90             88
1      Bob   30      M          92             88             85
2  Charlie   35      M          78             82             90
3    David   40      M          95             96             92
4      Eve   45      F          80             85             78

   History_Score  Total_Score
0             85          348
1             82          347
2             88          338
3             90          373
4             92          335
```

Calculating Summary Statistics Let's calculate summary statistics for the scores:

```
summary_stats = df[['Math_Score', 'Science_Score', 'English_Score',
'History_Score']].describe()
summary_stats
```

```
       Math_Score  Science_Score  English_Score  History_Score
count    5.000000       5.000000       5.000000       5.000000
mean    86.000000      88.200000      86.600000      87.400000
std      7.382412       5.310367       5.458938       3.974921
min     78.000000      82.000000      78.000000      82.000000
25%     80.000000      85.000000      85.000000      85.000000
50%     85.000000      88.000000      88.000000      88.000000
75%     92.000000      90.000000      90.000000      90.000000
max     95.000000      96.000000      92.000000      92.000000
```

Reshaping the Layout of Tables Let's reshape the DataFrame to have subjects as rows and scores as columns:

```
reshaped_data = df.melt(id_vars=['Name', 'Age', 'Gender'],
var_name='Subject', value_name='Score')
reshaped_data
```

	Name	Age	Gender	Subject	Score
0	Alice	25	F	Math_Score	85
1	Bob	30	M	Math_Score	92
2	Charlie	35	M	Math_Score	78
3	David	40	M	Math_Score	95
4	Eve	45	F	Math_Score	80
5	Alice	25	F	Science_Score	90
6	Bob	30	M	Science_Score	88
7	Charlie	35	M	Science_Score	82
8	David	40	M	Science_Score	96
9	Eve	45	F	Science_Score	85
10	Alice	25	F	English_Score	88
11	Bob	30	M	English_Score	85
12	Charlie	35	M	English_Score	90
13	David	40	M	English_Score	92
14	Eve	45	F	English_Score	78
15	Alice	25	F	History_Score	85
16	Bob	30	M	History_Score	82
17	Charlie	35	M	History_Score	88
18	David	40	M	History_Score	90
19	Eve	45	F	History_Score	92
20	Alice	25	F	Total_Score	348
21	Bob	30	M	Total_Score	347
22	Charlie	35	M	Total_Score	338
23	David	40	M	Total_Score	373
24	Eve	45	F	Total_Score	335

Combining Data from Multiple Tables Let's create another DataFrame with attendance information and merge it with the original DataFrame:

```
attendance_data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'David', 'Eve'],
    'Attendance': ['Present', 'Absent', 'Present', 'Present',
'Absent']
}

df_attendance = pd.DataFrame(attendance_data)

merged_data = pd.merge(df, df_attendance, on='Name')
merged_data
```

	Name	Age	Gender	Math_Score	Science_Score	English_Score	\
0	Alice	25	F	85	90	88	
1	Bob	30	M	92	88	85	
2	Charlie	35	M	78	82	90	
3	David	40	M	95	96	92	

```
4      Eve   45      F           80              85               78

   History_Score  Total_Score Attendance
0             85          348    Present
1             82          347     Absent
2             88          338    Present
3             90          373    Present
4             92          335     Absent
```

Handling Time Series Data with Ease Let's create a time series DataFrame with dates and corresponding scores:

```
dates = pd.date_range('2022-01-01', periods=5)
scores_data = {
    'Date': dates,
    'Math_Score': [85, 92, 78, 95, 80]
}

df_time_series = pd.DataFrame(scores_data)
df_time_series

        Date  Math_Score
0 2022-01-01          85
1 2022-01-02          92
2 2022-01-03          78
3 2022-01-04          95
4 2022-01-05          80
```

Manipulating Textual Data Let's create a new column called "Grade" based on the total score:

```
def calculate_grade(score):
    if score >= 350:
        return 'A'
    elif score >= 300:
        return 'B'
    elif score >= 250:
        return 'C'
    else:
        return 'D'

df['Grade'] = df['Total_Score'].apply(calculate_grade)
df

      Name  Age Gender  Math_Score  Science_Score  English_Score  \
0    Alice   25      F          85             90             88
1      Bob   30      M          92             88             85
2  Charlie   35      M          78             82             90
3    David   40      M          95             96             92
4      Eve   45      F          80             85             78
```

```
   History_Score  Total_Score Grade
0             85          348     B
1             82          347     B
2             88          338     B
3             90          373     A
4             92          335     B
```

Filtering Data Based on Multiple Conditions Let's filter the DataFrame to include only male students who scored above 85 in Math:

```
filtered_data = df[(df['Gender'] == 'M') & (df['Math_Score'] > 85)]
filtered_data
```

```
    Name  Age Gender  Math_Score  Science_Score  English_Score
History_Score  \
1    Bob   30      M          92             88             85
82
3  David   40      M          95             96             92
90

   Total_Score Grade
1          347     B
3          373     A
```

Creating a New Column Based on Conditions Let's create a new column called "Pass_Status" based on the total score:

```
df['Pass_Status'] = np.where(df['Total_Score'] >= 250, 'Pass', 'Fail')
df
```

```
      Name  Age Gender  Math_Score  Science_Score  English_Score  \
0    Alice   25      F          85             90             88
1      Bob   30      M          92             88             85
2  Charlie   35      M          78             82             90
3    David   40      M          95             96             92
4      Eve   45      F          80             85             78

   History_Score  Total_Score Grade Pass_Status
0             85          348     B        Pass
1             82          347     B        Pass
2             88          338     B        Pass
3             90          373     A        Pass
4             92          335     B        Pass
```

Calculating Group-Wise Summary Statistics Let's calculate the mean scores for each subject grouped by gender:

```
grouped_stats = df.groupby('Gender').agg({
    'Math_Score': 'mean',
    'Science_Score': 'mean',
    'English_Score': 'mean',
    'History_Score': 'mean'
})
grouped_stats
```

```
        Math_Score  Science_Score  English_Score  History_Score
Gender
F        82.500000      87.500000           83.0      88.500000
M        88.333333      88.666667           89.0      86.666667
```

Sorting Data Let's sort the DataFrame by total score in descending order:

```
sorted_data = df.sort_values(by='Total_Score', ascending=False)
sorted_data
```

```
      Name  Age Gender  Math_Score  Science_Score  English_Score  \
3    David   40      M          95             96             92
0    Alice   25      F          85             90             88
1      Bob   30      M          92             88             85
2  Charlie   35      M          78             82             90
4      Eve   45      F          80             85             78

   History_Score  Total_Score Grade Pass_Status
3             90          373     A        Pass
0             85          348     B        Pass
1             82          347     B        Pass
2             88          338     B        Pass
4             92          335     B        Pass
```

Concatenating DataFrames Let's create another DataFrame with additional student information and concatenate it with the original DataFrame:

```
additional_data = {
    'Name': ['Fiona', 'George'],
    'Age': [50, 55],
    'Gender': ['F', 'M'],
    'Math_Score': [78, 85],
    'Science_Score': [80, 88],
    'English_Score': [82, 90],
    'History_Score': [85, 88]
}

df_additional = pd.DataFrame(additional_data)

concatenated_data = pd.concat([df, df_additional], ignore_index=True)
concatenated_data
```

```
      Name  Age Gender  Math_Score  Science_Score  English_Score  \
0    Alice   25      F          85             90             88
1      Bob   30      M          92             88             85
2  Charlie   35      M          78             82             90
3    David   40      M          95             96             92
4      Eve   45      F          80             85             78
5    Fiona   50      F          78             80             82
6   George   55      M          85             88             90

   History_Score  Total_Score Grade Pass_Status
0             85        348.0     B        Pass
1             82        347.0     B        Pass
2             88        338.0     B        Pass
3             90        373.0     A        Pass
4             92        335.0     B        Pass
5             85          NaN   NaN         NaN
6             88          NaN   NaN         NaN
```

Handling Missing Data Let's introduce some missing data in the English scores and fill it with the mean score:

```
df.loc[2, 'English_Score'] = np.nan  # Introducing missing data

df_filled = df.fillna(value={'English_Score':
df['English_Score'].mean()})
df_filled
```

```
      Name  Age Gender  Math_Score  Science_Score  English_Score  \
0    Alice   25      F          85             90          88.00
1      Bob   30      M          92             88          85.00
2  Charlie   35      M          78             82          85.75
3    David   40      M          95             96          92.00
4      Eve   45      F          80             85          78.00

   History_Score  Total_Score Grade Pass_Status
0             85          348     B        Pass
1             82          347     B        Pass
2             88          338     B        Pass
3             90          373     A        Pass
4             92          335     B        Pass
```

Students practice question Step 1: Creating a Dummy Dataset Let's create a DataFrame with dummy data representing employee information:

```
import pandas as pd
import numpy as np

# Creating a dummy dataset
data = {
```

```
    'Name': ['Alice', 'Bob', 'Charlie', 'David', 'Eve'],
    'Age': [25, 30, 35, 40, 45],
    'Salary': [50000, 60000, 70000, 80000, 90000],
    'Department': ['HR', 'IT', 'Finance', 'HR', 'IT'],
    'Start_Date': pd.to_datetime(['2020-01-01', '2019-03-15', '2021-
05-20', '2018-09-10', '2022-02-28']),
    'Experience': [5, 10, 3, 15, 2],
    'Rating': [4.2, 3.8, 4.5, 4.0, 4.7]
}

df = pd.DataFrame(data)
df
```

```
      Name  Age  Salary Department Start_Date  Experience  Rating
0    Alice   25   50000         HR 2020-01-01           5     4.2
1      Bob   30   60000         IT 2019-03-15          10     3.8
2  Charlie   35   70000    Finance 2021-05-20           3     4.5
3    David   40   80000         HR 2018-09-10          15     4.0
4      Eve   45   90000         IT 2022-02-28           2     4.7
```

Step 2: Selecting a Subset of the DataFrame Let's select employees who are older than 30:

```
subset = df[df['Age'] > 30]
subset
```

```
      Name  Age  Salary Department Start_Date  Experience  Rating
2  Charlie   35   70000    Finance 2021-05-20           3     4.5
3    David   40   80000         HR 2018-09-10          15     4.0
4      Eve   45   90000         IT 2022-02-28           2     4.7
```

Step 3: Creating New Columns Derived from Existing Columns Let's create a new column called "Age_Group" based on the age of the employees:

```
df['Age_Group'] = np.where(df['Age'] < 30, 'Young', 'Old')
df
```

```
      Name  Age  Salary Department Start_Date  Experience  Rating
Age_Group
0    Alice   25   50000         HR 2020-01-01           5     4.2
Young
1      Bob   30   60000         IT 2019-03-15          10     3.8
Old
2  Charlie   35   70000    Finance 2021-05-20           3     4.5
Old
3    David   40   80000         HR 2018-09-10          15     4.0
Old
4      Eve   45   90000         IT 2022-02-28           2     4.7
Old
```

Step 4: Calculating Summary Statistics Let's calculate summary statistics for the numerical columns in the DataFrame:

```
summary_stats = df.describe()
summary_stats
```

```
              Age        Salary           Start_Date  Experience
Rating
count   5.000000      5.000000                    5     5.00000
5.000000
mean   35.000000  70000.000000  2020-04-14 19:12:00     7.00000
4.240000
min    25.000000  50000.000000  2018-09-10 00:00:00     2.00000
3.800000
25%    30.000000  60000.000000  2019-03-15 00:00:00     3.00000
4.000000
50%    35.000000  70000.000000  2020-01-01 00:00:00     5.00000
4.200000
75%    40.000000  80000.000000  2021-05-20 00:00:00    10.00000
4.500000
max    45.000000  90000.000000  2022-02-28 00:00:00    15.00000
4.700000
std     7.905694  15811.388301                  NaN     5.43139
0.364692
```

Step 5: Reshaping the Layout of Tables Let's reshape the DataFrame to have "Name" as the index and "Department" as columns, with "Salary" as values:

```
reshaped_data = df.pivot(index='Name', columns='Department',
values='Salary')
reshaped_data
```

```
Department  Finance       HR       IT
Name
Alice           NaN  50000.0      NaN
Bob             NaN      NaN  60000.0
Charlie     70000.0      NaN      NaN
David           NaN  80000.0      NaN
Eve             NaN      NaN  90000.0
```

Step 6: Combining Data from Multiple Tables Let's create another DataFrame with bonus information and merge it with the original DataFrame:

```
bonus_data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'David', 'Eve'],
    'Bonus': [2000, 3000, 4000, 5000, 6000]
}

df_bonus = pd.DataFrame(bonus_data)
```

```
merged_data = pd.merge(df, df_bonus, on='Name')
merged_data
```

```
      Name  Age  Salary Department Start_Date  Experience  Rating
Age_Group  \
0    Alice   25   50000         HR 2020-01-01           5     4.2
Young
1      Bob   30   60000         IT 2019-03-15          10     3.8
Old
2  Charlie   35   70000    Finance 2021-05-20           3     4.5
Old
3    David   40   80000         HR 2018-09-10          15     4.0
Old
4      Eve   45   90000         IT 2022-02-28           2     4.7
Old

   Bonus
0   2000
1   3000
2   4000
3   5000
4   6000
```

Step 7: Handling Time Series Data with Ease Let's resample the data to a monthly frequency and calculate the mean salary for each month:

Step 8: Manipulating Textual Data Let's create a new column based on the length of the employee's name:

```
df['Name_Length'] = df['Name'].apply(len)
df
```

```
               Name  Age  Salary Department  Experience  Rating
Age_Group  \
Start_Date

2020-01-01    Alice   25   50000         HR           5     4.2
Young
2019-03-15      Bob   30   60000         IT          10     3.8
Old
2021-05-20  Charlie   35   70000    Finance           3     4.5
Old
2018-09-10    David   40   80000         HR          15     4.0
Old
2022-02-28      Eve   45   90000         IT           2     4.7
Old

            Name_Length
Start_Date
2020-01-01            5
```

```
2019-03-15             3
2021-05-20             7
2018-09-10             5
2022-02-28             3
```

Step 9: Filtering Data Based on Multiple Conditions Let's filter the DataFrame to include only employees from the IT department who are older than 30:

filtered_data = df[(df['Department'] == 'IT') & (df['Age'] > 30)] filtered_data

Step 10: Creating a New Column Based on Conditions Let's create a new column called "Performance" based on the employee's rating:

```
df['Performance'] = np.where(df['Rating'] >= 4.0, 'Good', 'Average')
df
```

```
               Name  Age  Salary Department  Experience  Rating
Age_Group  \
Start_Date

2020-01-01    Alice   25   50000         HR           5     4.2
Young
2019-03-15      Bob   30   60000         IT          10     3.8
Old
2021-05-20  Charlie   35   70000    Finance           3     4.5
Old
2018-09-10    David   40   80000         HR          15     4.0
Old
2022-02-28      Eve   45   90000         IT           2     4.7
Old

            Name_Length Performance
Start_Date
2020-01-01            5        Good
2019-03-15            3     Average
2021-05-20            7        Good
2018-09-10            5        Good
2022-02-28            3        Good
```

Step 11: Calculating Group-Wise Summary Statistics Let's calculate the mean salary and experience for each department:

```
grouped_stats = df.groupby('Department').agg({'Salary': 'mean',
'Experience': 'mean'})
grouped_stats
```

```
              Salary  Experience
Department
Finance     70000.0         3.0
```

```
HR          65000.0         10.0
IT          75000.0          6.0
```

Step 12: Sorting Data Let's sort the DataFrame by age in descending order:

```
sorted_data = df.sort_values(by='Age', ascending=False)
sorted_data
```

```
                Name  Age  Salary Department  Experience  Rating
Age_Group  \
Start_Date

2022-02-28       Eve   45   90000         IT           2     4.7
Old
2018-09-10     David   40   80000         HR          15     4.0
Old
2021-05-20   Charlie   35   70000    Finance           3     4.5
Old
2019-03-15       Bob   30   60000         IT          10     3.8
Old
2020-01-01     Alice   25   50000         HR           5     4.2
Young

             Name_Length Performance
Start_Date
2022-02-28             3        Good
2018-09-10             5        Good
2021-05-20             7        Good
2019-03-15             3     Average
2020-01-01             5        Good
```

Step 13: Concatenating DataFrames Let's create a new DataFrame with additional employee information and concatenate it with the original DataFrame:

```
additional_data = {
    'Name': ['Fiona', 'George'],
    'Age': [50, 55],
    'Salary': [95000, 100000],
    'Department': ['Finance', 'IT'],
    'Start_Date': pd.to_datetime(['2015-04-01', '2016-08-20']),
    'Experience': [20, 25],
    'Rating': [4.9, 4.3]
}

df_additional = pd.DataFrame(additional_data)

concatenated_data = pd.concat([df, df_additional], ignore_index=True)
concatenated_data
```

```
      Name  Age  Salary Department  Experience  Rating Age_Group
Name_Length  \
0    Alice   25   50000         HR           5     4.2     Young
5.0
1      Bob   30   60000         IT          10     3.8       Old
3.0
2  Charlie   35   70000    Finance           3     4.5       Old
7.0
3    David   40   80000         HR          15     4.0       Old
5.0
4      Eve   45   90000         IT           2     4.7       Old
3.0
5    Fiona   50   95000    Finance          20     4.9       NaN
NaN
6   George   55  100000         IT          25     4.3       NaN
NaN

  Performance Start_Date
0        Good        NaT
1     Average        NaT
2        Good        NaT
3        Good        NaT
4        Good        NaT
5         NaN 2015-04-01
6         NaN 2016-08-20
```

Step 14: Handling Missing Data Let's introduce some missing data and fill it with the mean salary:

```
df.loc[2, 'Salary'] = np.nan  # Introducing missing data

df_filled = df.fillna(value={'Salary': df['Salary'].mean()})
df_filled
```

```
                       Name   Age   Salary Department  Experience
Rating  \
Start_Date

2020-01-01 00:00:00    Alice  25.0  50000.0         HR         5.0
4.2
2019-03-15 00:00:00      Bob  30.0  60000.0         IT        10.0
3.8
2021-05-20 00:00:00  Charlie  35.0  70000.0    Finance         3.0
4.5
2018-09-10 00:00:00    David  40.0  80000.0         HR        15.0
4.0
2022-02-28 00:00:00      Eve  45.0  90000.0         IT         2.0
4.7
2                        NaN   NaN  70000.0        NaN         NaN
```

```
NaN

                     Age_Group  Name_Length Performance
Start_Date
2020-01-01 00:00:00     Young          5.0        Good
2019-03-15 00:00:00       Old          3.0     Average
2021-05-20 00:00:00       Old          7.0        Good
2018-09-10 00:00:00       Old          5.0        Good
2022-02-28 00:00:00       Old          3.0        Good
2                         NaN          NaN         NaN
```